



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema: Árboles
Unidad de Organización Curricular: BÁSICA
Nivel y Paralelo: Tercero - B
Nombre: Medina Chico Darlyn Monserrath
.....
Asignatura: Estructura de Datos
Docente: Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en el trabajo con árboles.

Específicos:

- Implementar operaciones fundamentales sobre árboles N-arios y árboles binarios utilizando recursividad.
- Aplicar recorridos (In-Order) y transformaciones (inversión) para manipular la estructura de árboles.
- Diferenciar el comportamiento y aplicación de árboles N-arios vs Árboles Binarios de Búsqueda (BST).

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- PC de escritorio o portátil con JDK instalado e IDE de su elección.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

2.7 Resultados obtenidos

Se desarrollan habilidades en la solución a un problema de baja complejidad utilizando árboles.



A continuación se presentan los 5 ejercicios implementados en lenguaje C++ y Java, junto con sus respectivos códigos, árboles de prueba y salidas obtenidas.

Ejercicio 1: Conteo de nodos en árbol N-ario

Descripción: Implementar una función que recorra recursivamente un árbol N-ario y cuente el número total de nodos.

Código implementado (C++):

```
int contarNodos(NodoN* raiz) {
    if (raiz == nullptr) return 0;
    int total = 1;
    for (NodoN* hijo : raiz->hijos) {
        total += contarNodos(hijo);
    }
    return total;
}
```

Código implementado (Java):

```
public static int contarNodos(NodoN raiz) {
    if (raiz == null) return 0;
    int total = 1;
    for (NodoN hijo : raiz.hijos) {
        total += contarNodos(hijo);
    }
    return total;
}
```

Captura de ejecución:

```
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Es
tructura de Datos\PARCIAL - 2\Grupo4_Arboles\Guia_Ape3_Arboles>
```

Ejercicio 2: Inserción en Árbol Binario de Búsqueda (BST)

Descripción: Implementar inserción en BST respetando la propiedad: valores menores a la izquierda, mayores o iguales a la derecha.

Código implementado (C++):

```
Nodo* insertar(Nodo* raiz, int valor) {
    if (raiz == nullptr) return new Nodo(valor);
    if (valor < raiz->valor)
        raiz->izquierdo = insertar(raiz->izquierdo, valor);
    else
        raiz->derecho = insertar(raiz->derecho, valor);
    return raiz;
}
```

Código implementado (Java):



```
public static Nodo insertar(Nodo raiz, int valor) {  
    if (raiz == null) return new Nodo(valor);  
    if (valor < raiz.valor)  
        raiz.izquierdo = insertar(raiz.izquierdo, valor);  
    else  
        raiz.derecho = insertar(raiz.derecho, valor);  
    return raiz;  
}
```

Captura de ejecución:

```
--- Prueba Ejercicio 2 ---  
Raiz (Esperado 10): 10  
Hijo Izquierdo de Raiz (Esperado 5): 5  
Hijo Derecho de Raiz (Esperado 15): 15  
Hijo Izquierdo del 5 (Esperado 3): 3  
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\Grupo4_Arboles\Guia_Ape3_Arboles>
```

Ejercicio 3: Cálculo de profundidad máxima (altura)

Descripción: Calcular la altura del árbol (número de nodos en la ruta más larga desde raíz hasta hoja).

Código implementado (C++):

```
int calcularAltura(Nodo* raiz) {  
    if (raiz == nullptr) return 0;  
    int alturaIzq = calcularAltura(raiz->izquierdo);  
    int alturaDer = calcularAltura(raiz->derecho);  
    return 1 + max(alturaIzq, alturaDer);  
}
```

Código implementado (Java):

```
public static int calcularAltura(Nodo raiz) {  
    if (raiz == null) return 0;  
    int alturaIzq = calcularAltura(raiz.izquierdo);  
    int alturaDer = calcularAltura(raiz.derecho);  
    return 1 + Math.max(alturaIzq, alturaDer);  
}
```

Captura de ejecución:



```
es> & C:\Program Files\Eclipse Adoptium\jdk-21.0.11-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Usuario\AppData\Roaming\Code\User\workspaceStorage\08e97672aa4746c61708897f59c62d9\redhat.java\jdt_ws\Guia_Ape3_Arboles_7d8ae5d\bin' 'Ejercicio3_Binario2'
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
Altura de árbol nulo (esperado 0): 0
```

Ejercicio 4: Recorrido In-Order

Descripción: Implementar recorrido in-order (izquierda → nodo → derecha) que produce elementos ordenados en un BST.

Código implementado (C++):

```
inOrderAux(Nodo* nodo, vector<int>& resultado) {
    if (nodo == nullptr) return;
    inOrderAux(nodo->izquierdo, resultado);
    resultado.push_back(nodo->valor);
    inOrderAux(nodo->derecho, resultado);
}
```

Código implementado (Java):

```
public static void inOrderAux(Nodo nodo, List<Integer> resultado) {
    if (nodo == null) return;
    inOrderAux(nodo.izquierdo, resultado);
    resultado.add(nodo.valor);
    inOrderAux(nodo.derecho, resultado);
}
```

Captura de ejecución:

```
--- Prueba Ejercicio 4 ---
Resultado esperado: [1, 2, 3, 4, 5, 6, 7]
Tu resultado:      [1, 2, 3, 4, 5, 6, 7]
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\Grupo4_Arboles\Guia_Ape3_Arboles>
```

Ejercicio 5: Inversión del árbol (árbol espejo)

Descripción: Transformar el árbol intercambiando recursivamente los hijos izquierdo y derecho de cada nodo.

Código implementado (C++):

```
Nodo* invertir(Nodo* raiz) {
    if (raiz == nullptr) return nullptr;
    Nodo* temp = invertir(raiz->izquierdo);
```



```
raiz->izquierdo = invertir(raiz->derecho);  
raiz->derecho = temp;  
return raiz;  
}
```

Código implementado (Java):

```
public static Nodo invertir(Nodo raiz) {  
    if (raiz == null) return null;  
    Nodo temp = invertir(raiz.izquierdo);  
    raiz.izquierdo = invertir(raiz.derecho);  
    raiz.derecho = temp;  
    return raiz;  
}
```

Captura de ejecución:

```
72aa4746c61708897f59c62d9\redhat.java\jdt_ws\Guia_Ape3_Arboles_73  
d8ae5d\bin' 'Ejercicio5_Transformacion'  
--- Prueba Ejercicio 5 ---  
Antes de invertir:  
Hijo Izq: 2 | Hijo Der: 3  
  
Después de invertir (Esperado: Izq 3 | Der 2):  
Hijo Izq: 3 | Hijo Der: 2
```

Análisis de Resultados

Los 5 ejercicios fueron implementados exitosamente tanto en C++ como en Java. Se desarrollaron habilidades fundamentales en el trabajo con arboles:

- Recursividad: Todos los ejercicios utilizaron recursion como tecnica principal para recorrer y transformar los arboles, demostrando que es la aproximacion mas natural para estructuras jerarquicas.
- Arboles N-arios vs Binarios: Se evidencio la diferencia estructural fundamental:
 - Arbol N-ario: cada nodo contiene un vector de hijos
 - Arbol binario: cada nodo tiene maximo 2 hijos (izquierdo y derecho)
- Propiedad BST: La insercion en BST respeta la propiedad de orden, permitiendo busquedas eficientes en $O(\log n)$.
- Recorrido In-Order: Sobre un BST produce automaticamente una secuencia ordenada de menor a mayor.
- Transformacion estructural: La inversion modifica completamente la forma del arbol intercambiando recursivamente los punteros de los hijos.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico



- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

Las estructuras dinámicas de árboles constituyen una alternativa al procesamiento secuencial de los datos, de amplia aplicación práctica.

2.10 Recomendaciones

- Tener en cuenta las condiciones específicas de cada situación para decidir qué estructura de datos utilizar
- Las estructuras dinámicas de árboles permiten organizar datos jerárquicamente y procesarlos de forma recursiva.
- La recursividad es una herramienta poderosa para recorrer y transformar árboles.

- El BST facilita búsquedas eficientes si se mantiene la propiedad de orden.
- El recorrido in-order sobre un BST produce una secuencia ordenada.

2.11 Referencias bibliográficas

Insertar las referencias bibliográficas empleadas aplicando la norma IEEE.

2.12 Link de Github

https://github.com/Monse1806/Guia_Ape3_Arboles.git